

Theoretische Informatik HS26

Marius Tomek

Übungsstunde 02

30. September 2026

ETH Zürich

marius.tomek@asvz.ch

Kontakt

In der Übungsstunde

Per Mail an marius.tomek@asvz.ch

Slides

Webseite: <https://mariustomek.github.io/TI>

Nächste woche

Ich bin in weg, daher Übung in CHN D 46 bei Philip

Aufgaben: Gerne mehr Gruppen!

- ① Letzte Woche & Feedback zur Serie
- ② Algorithmen und Entscheidungsprobleme
- ③ Endliche Automaten - Einführung

Letzte Woche & Feedback zur Serie

- I. Grundsätzlich gut.
- II. Bei manchen die Länge reduzieren und konkreter argumentieren.
- III. Bei anderen hat Argumentation gefehlt. Steht in der Korrektur.
- IV. Bei Aufgabe 3: k und l beliebig. Alle Antworten müssen begründet werden.

Algorithmen und Entscheidungsprobleme

Vorläufige Definition des Begriffs Algorithmus

Mathematische Definition folgt in Kapitel 4 (Turingmaschinen).

Vorerst betrachten wir Programme, die **für jede zulässige Eingabe halten und eine Ausgabe liefern**, als Algorithmen.

Wir betrachten ein Programm (Algorithmus) A als Abbildung $A : \Sigma_1^* \rightarrow \Sigma_2^*$ für beliebige Alphabete Σ_1 und Σ_2 . Dies bedeutet, dass

- (i) die Eingaben als Wörter über Σ_1 kodiert sind,
- (ii) die Ausgaben als Wörter über Σ_2 kodiert sind und
- (iii) A für jede Eingabe eine eindeutige Ausgabe bestimmt.

A und B äquivalent $\iff$ Eingabealphabet Σ gleich, $A(x) = B(x), \forall x \in \Sigma^*$

Ie. diese Notion von "Äquivalenz" bezieht sich nur auf die Ein und Ausgabe.

Algorithmen generieren Wörter

Sei Σ ein Alphabet und $x \in \Sigma^*$. Wir sagen, dass ein Algorithmus A das Wort x **generiert**, falls A für die Eingabe λ die Ausgabe x liefert.

Beispiel:

```
 $A_n$ :      begin
              for  $i = 1$  to  $n$ ;
                write (01);
              end
```

A_n generiert $(01)^n$.

Sei Σ ein Alphabet und sei $L \subseteq \Sigma^*$. A ist ein **Aufzählungsalgorithmus** für L , falls A für jede Eingabe $n \in \mathbb{N} \setminus \{0\}$ die Wortfolge $x_1, \dots, x_n$ ausgibt, wobei $x_1, \dots, x_n$ die kanonisch n ersten Wörter in L sind.

Das **Entscheidungsproblem** (Σ, L) für ein gegebenes Alphabet Σ und eine gegebene Sprache $L \subseteq \Sigma^*$ ist, für jedes $x \in \Sigma^*$ zu entscheiden, ob

$$x \in L \text{ oder } x \notin L.$$

Ein Algorithmus A **löst** das Entscheidungsproblem (Σ, L) , falls für alle $x \in \Sigma^*$ gilt:

$$A(x) = \begin{cases} 1, & \text{falls } x \in L, \\ 0, & \text{falls } x \notin L. \end{cases}$$

Wir sagen auch, dass A die Sprache L erkennt.

Aufgabe 2.21 - machen wir am Ende

Wenn für eine Sprache L ein Algorithmus existiert, der L erkennt, sagen wir, dass L **rekursiv** ist.

$$L \in \mathcal{L}_R$$

Aufgabe 2.21

Beweisen Sie, dass eine Sprache L genau dann rekursiv ist, wenn ein Aufzählungsalgorithmus für L existiert.

Das **Entscheidungsproblem** (Σ, L) für ein gegebenes Alphabet Σ und eine gegebene Sprache $L \subseteq \Sigma^*$ ist, für jedes $x \in \Sigma^*$ zu entscheiden, ob

$$x \in L \text{ oder } x \notin L.$$

Ein Algorithmus A **löst** das Entscheidungsproblem (Σ, L) , falls für alle $x \in \Sigma^*$ gilt:

$$A(x) = \begin{cases} 1, & \text{falls } x \in L, \\ 0, & \text{falls } x \notin L. \end{cases}$$

Wir sagen auch, dass A die Sprache L erkennt.

Endliche Automaten - Einführung

Ein (deterministischer) **endlicher Automat (EA)** ist ein Quintupel $M = (Q, \Sigma, \delta, q_0, F)$, wobei

- (i) Q eine endliche Menge von **Zuständen** ist,
- (ii) Σ ein Alphabet, genannt **Eingabealphabet**, ist,
- (iii) $q_0 \in Q$ der Anfangszustand ist,
- (iv) $F \subseteq Q$ die **Menge der akzeptierenden Zustände** ist und
- (v) $\delta : Q \times \Sigma \rightarrow Q$ die **Übergangsfunktion** ist.

Konfigurationen

Eine **Konfiguration** von M ist ein Tupel $(q, w) \in Q \times \Sigma^*$.

- "M befindet sich in einer Konfiguration $(q, w) \in Q \times \Sigma^*$, wenn M im Zustand q ist und noch das Suffix w eines Eingabewortes lesen soll."
- Die Konfiguration $(q_0, x) \in \{q_0\} \times \Sigma^*$ heisst die **Startkonfiguration von M auf x** .
- Jede Konfiguration aus $Q \times \{\lambda\}$ nennt man **Endkonfiguration**.

Ein **Schritt** von M ist eine Relation (auf Konfigurationen) $\mid_M \subseteq (Q \times \Sigma^*) \times (Q \times \Sigma^*)$, definiert durch

$$(q, w) \mid_M (p, x) \iff w = ax, a \in \Sigma \text{ und } \delta(q, a) = p.$$

Berechnungen

Eine **Berechnung** C von M ist eine endliche Folge $C = C_0, C_1, \dots, C_n$ von Konfigurationen, so dass

$$C_i \mid_M C_{i+1} \text{ für alle } 0 \leq i \leq n-1.$$

C ist die **Berechnung von M auf einer Eingabe $x \in \Sigma^*$** , falls $C_0 = (q_0, x)$ und $C_n \in Q \times \{\lambda\}$ eine Endkonfiguration ist.

Falls $C_n \in F \times \{\lambda\}$, sagen wir, dass C eine **akzeptierende Berechnung** von M auf x ist, und dass M **das Wort x akzeptiert**.

Falls $C_n \in (Q \setminus F) \times \{\lambda\}$, sagen wir, dass C eine **verwerfende Berechnung** von M auf x ist, und dass M **das Wort x verwirft (nicht akzeptiert)**.

Sei $M = (Q, \Sigma, \delta, q_0, F)$ ein endlicher Automat. Wir definieren $\mid_M^*$ als die reflexive und transitive Hülle der Schrittrelation $\mid_M$ von M ; daher ist

$$(q, w) \mid_M^* (p, u) \iff (q = p \wedge w = u) \text{ oder } \exists k \in \mathbb{N} \setminus \{0\},$$

so dass

- (i) $w = a_1 a_2 \dots a_k u, a_i \in \Sigma$ für $i = 1, 2, \dots, k$, und
- (ii) $\exists r_1, r_2, \dots, r_{k-1} \in Q$, so dass

$$(q, w) \mid_M (r_1, a_2 \dots a_k u) \mid_M \dots \mid_M (r_{k-1}, a_k u) \mid_M (p, u)$$

Wir definieren $\hat{\delta} : Q \times \Sigma^* \rightarrow Q$ durch:

- (i) $\hat{\delta}(q, \lambda) = q$ für alle $q \in Q$ und
- (ii) $\hat{\delta}(q, wa) = \delta(\hat{\delta}(q, w), a)$ für alle $a \in \Sigma, w \in \Sigma^*, q \in Q$.

$$\hat{\delta}(q, w) = p \iff (q, w) \mid_M^* (p, \lambda)$$

Die **von M akzeptierte Sprache** $L(M)$ ist definiert als

$$\begin{aligned} L(M) &= \{w \in \Sigma^* \mid \text{Berechnung von } M \text{ auf } w \text{ endet in } (p, \lambda) \in F \times \{\lambda\}\} \\ &= \{w \in \Sigma^* \mid (q_0, w) \xrightarrow{*}_M (p, \lambda) \wedge p \in F\} \\ &= \{w \in \Sigma^* \mid \hat{\delta}(q_0, w) \in F\} \end{aligned}$$

$\mathcal{L}_{\text{EA}} = \{L(M) \mid M \text{ ist ein EA}\}$ ist die Klasse der Sprachen, die von endlichen Automaten akzeptiert werden.

$\mathcal{L}_{\text{EA}}$ bezeichnet man auch als die **Klasse der regulären Sprachen**, und jede Sprache $L \in \mathcal{L}_{\text{EA}}$ wird **regulär** genannt.

WICHTIG: Unterschied zu rekursiv, später in der Vorlesung mehr dazu! $\mathcal{L}_{\text{EA}} \subsetneq \mathcal{L}_R$

Klassen für alle Zustände im Endlichen Automaten

Für alle $p \in Q$ definieren wir die Klasse

$$\begin{aligned}\mathbf{KI}[p] &= \{w \in \Sigma^* \mid \hat{\delta}(q_0, w) = p\} \\ &= \{w \in \Sigma^* \mid (q_0, w) \stackrel{*}{\mid}_M (p, \lambda)\}\end{aligned}$$

Wir bemerken dann

$$\bigcup_{q \in Q} \mathbf{KI}[q] = \Sigma^*$$

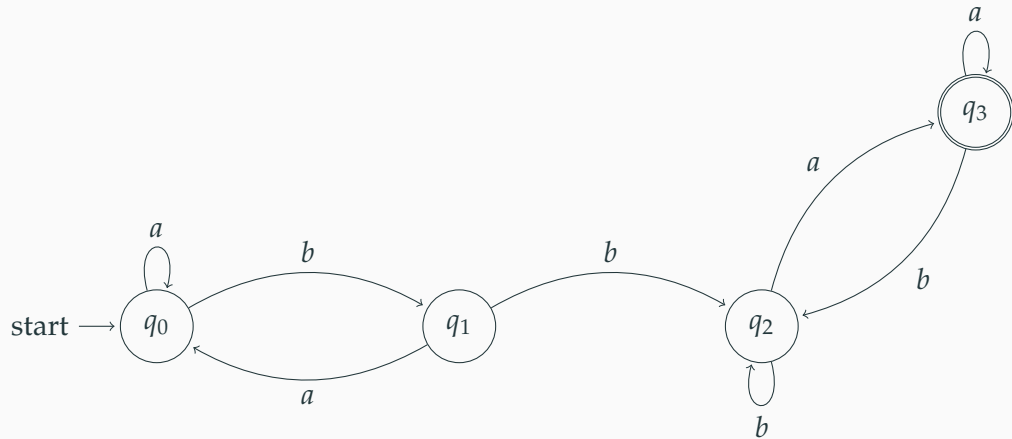
$$\mathbf{KI}[q] \cap \mathbf{KI}[p] = \emptyset, \forall p, q \in Q, p \neq q$$

$$L(M) = \bigcup_{q \in F} \mathbf{KI}[q]$$

Entwerfen sie für folgende Sprache einen Endlichen Automat und geben Sie eine Beschreibung von $Kl[q]$ für jeden Zustand $q \in Q$.

$$L_1 = \{xbbya \in \{a, b\}^* \mid x, y \in \{a, b\}^*\}$$

EA Konstruktion - Beispielaufgabe



Wir beschreiben nun die Klassen für die Zustände q_0, q_1, q_2, q_3 :

$$\text{Kl}[q_0] = \{wa \in \{a, b\}^* \mid \text{Das Wort } w \text{ enthält nicht die Teilfolge } bb\} \cup \{\lambda\}$$

$$\text{Kl}[q_1] = \{wb \in \{a, b\}^* \mid \text{Das Wort } w \text{ enthält nicht die Teilfolge } bb\}$$

$$\text{Kl}[q_3] = \{wa \in \{a, b\}^* \mid \text{Das Wort } w \text{ enthält die Teilfolge } bb\} = L_1$$

$$\text{Kl}[q_2] = \{a, b\}^* - (\text{Kl}[q_0] \cup \text{Kl}[q_1] \cup \text{Kl}[q_3])$$

Sei Σ ein Alphabet und seien $M_1 = (Q_1, \Sigma, \delta_1, q_{01}, F_1)$ und $M_2 = (Q_2, \Sigma, \delta_2, q_{02}, F_2)$ zwei EA. Für jede Mengenoperation $\odot \in \{\cup, \cap, -\}$ existiert ein EA M , so dass

$$L(M) = L(M_1) \odot L(M_2).$$

Sei $M = (Q, \Sigma, \delta, q_0, F_\odot)$, wobei

- (i) $Q = Q_1 \times Q_2$
- (ii) $q_0 = (q_{01}, q_{02})$
- (iii) für alle $q \in Q_1, p \in Q_2$ und $a \in \Sigma$, $\delta((q, p), a) = (\delta_1(q, a), \delta_2(p, a))$,
- (iv) falls $\odot = \cup$, dann ist $F = F_1 \times Q_2 \cup Q_1 \times F_2$
falls $\odot = \cap$, dann ist $F = F_1 \times F_2$, und
falls $\odot = -$, dann ist $F = F_1 \times (Q_2 - F_2)$.

Verwenden Sie die Methode des modularen Entwurfs (Konstruktion eines Produktautomaten), um einen endlichen Automaten (in Diagrammdarstellung) für die Sprache

$$L = \{w \in \{a, b\}^* \mid |w|_a = 2 \text{ oder } w = ya\}$$

zu entwerfen. Zeichnen Sie auch jeden der Teilautomaten und geben Sie für die Teilautomaten für jeden Zustand q die Klasse $\text{Kl}[q]$ an.

Wir teilen L wie folgt auf:

$$L = L_1 \cup L_2 \text{ wobei gilt:}$$

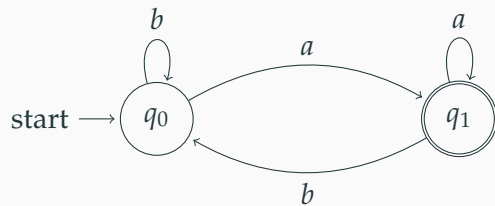
$$L_1 = \{w \in \{a, b\}^* \mid w = ya\}$$

$$L_2 = \{w \in \{a, b\}^* \mid |w|_a = 2\}$$

Zuerst zeichnen wir die 2 einzelnen Teilautomaten und geben für jeden Zustand q bzw. p die Klasse $Kl[q]$ respektive $Kl[p]$ an:

Produktautomat - Beispielaufgabe

erster Teilautomat: $L_1 = \{w \in \{a, b\}^* \mid w = ya\}$

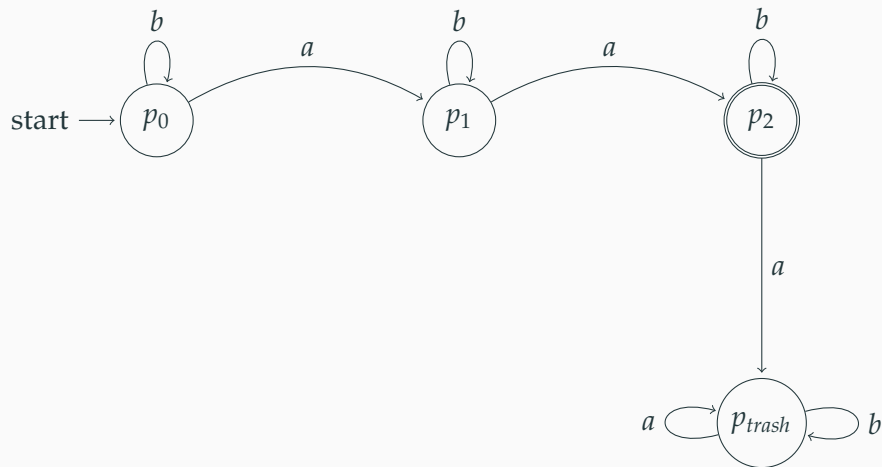


$$\text{KI}[q_1] = \{ya \mid y \in \{a, b\}^*\}$$

$$\text{KI}[q_0] = \{yb \mid y \in \{a, b\}^*\} \cup \{\lambda\}$$

Produktautomat - Beispielaufgabe

zweiter Teilautomat: $L_2 = \{w \in \{a, b\}^* \mid |w|_a = 2\}$



Wir beschreiben nun die Zustände für die Klassen p_0, p_1, p_2, p_{trash} :

$$Kl[p_0] = \{w \in \{a, b\}^* \mid |w|_a = 0\}$$

$$Kl[p_1] = \{w \in \{a, b\}^* \mid |w|_a = 1\}$$

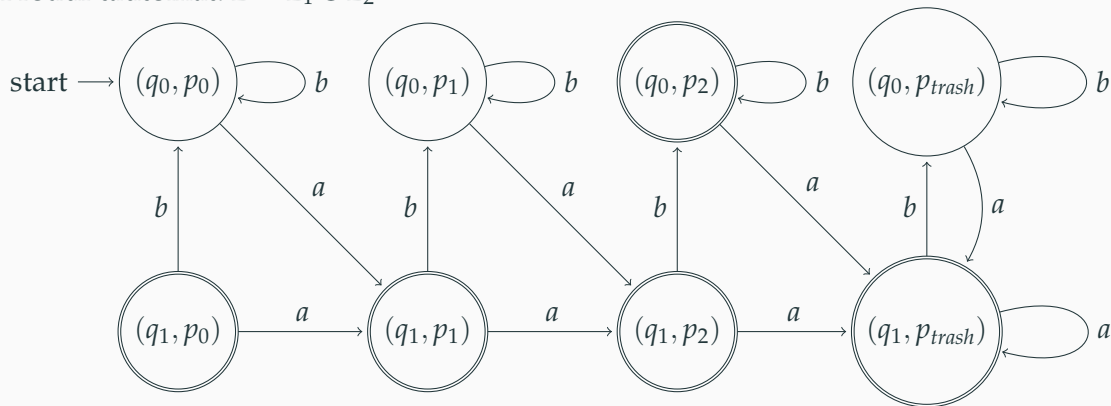
$$Kl[p_2] = \{w \in \{a, b\}^* \mid |w|_a = 2\}$$

$$Kl[p_{trash}] = \{w \in \{a, b\}^* \mid |w|_a > 2\}$$

Produktautomat - Beispielaufgabe

Zum Schluss kombinieren wir diese Teilautomaten zu einem Produktautomaten:

Produktautomat: $L = L_1 \cup L_2$



Aufgabe 2.21

Wenn für eine Sprache L ein Algorithmus existiert, der L erkennt, sagen wir, dass L **rekursiv** ist.

$$L \in \mathcal{L}_R$$

Aufgabe 2.21

Beweisen Sie, dass eine Sprache L genau dann rekursiv ist, wenn ein Aufzählungsalgorithmus für L existiert.

Aufgabe 2.21

L rekursiv ($\implies$) existiert Aufzählungsalgorithmus:

Sei A ein Algorithmus, der L erkennt. Wir beschreiben nun einen Aufzählungsalgorithmus B konstruktiv.

Algorithm 1 $B(\Sigma, n)$

$i \leftarrow 0$

while $i \leq n$ **do**

$w \leftarrow$ kanonisch nächstes Wort über Σ^*

if $A(w) = 1$ **then**

 print(w)

$i \leftarrow i + 1$

end if

end while

Aufgabe 2.21

Aufzählungsalgorithmus $B \implies L$ rekursiv:

Algorithm 2 $A(\Sigma, w)$

```
 $n \leftarrow |\Sigma|^{|w|+1}$   
 $L \leftarrow B(\Sigma, n)$   
if  $w \in L$  then  
    print(1)  
else  
    print(0)  
end if
```

Es gibt ein kleines Problem. B könnte unendlich lange laufen, falls $n > |L|$. Es sollte nicht so schwierig sein, B zu modifizieren, dass es die Berechnung aufhört, falls es keine weiteren Wörter in L gibt.